

Chapter 1: Foundations

The Windows NT line of operating systems has quite a bit of history, starting with version 3.1 launched in 1993. Today's Windows 10 is the latest successor to that initial NT 3.1. The fundamental concepts of current Windows systems is essentially the same as it was back in 1993. This shows the strength of the initial OS design. That said, Windows grew significantly since its inception, with many new features and enhancements to existing ones.

This book is about system programming, typically regarded as low-level programming of the operating system's core services, without which no significant work can be accomplished. System programming uses low level APIs to use and manipulate core objects and mechanisms in Windows, such as processes, threads and memory.

In this chapter, we'll take a look at the foundations of Windows system programming, starting from the core concepts and APIs.

In this chapter:

- **Windows Architecture Overview**
 - **Windows Application Development**
 - **Working with Strings**
 - **32-bit vs. 64-bit Development**
 - **Coding Conventions**
 - **Handling API Errors**
 - **The Windows Version**
-

Windows Architecture Overview

We'll start with a brief description of some core concepts and components in Windows. These will be elaborated on in the relevant chapters that follow.

Processes

A process is a containment and management object that represents a running instance of a program. The term “process runs” which is used fairly often, is inaccurate. Processes don’t run - processes manage. Threads are the ones that execute code and technically run. From a high-level perspective, a process owns the following:

- An executable program, which contains the initial code and data used to execute code within the process.
- A private virtual address space, used for allocating memory for whatever purposes the code within the process needs it.
- An access token (sometimes referred to as *primary token*), which is an object that stores the default security context of the process, used by threads executing code within the process (unless a thread assumes a different token by using impersonation).
- A private handle table to *Executive* (kernel) objects, such as events, semaphores, and files.
- One or more threads of execution. A normal user-mode process is created with one thread (executing the main entry point for the process). A user-mode process without threads is mostly useless and under normal circumstances will be destroyed by the kernel.

These elements of a process are depicted in figure 1-1.

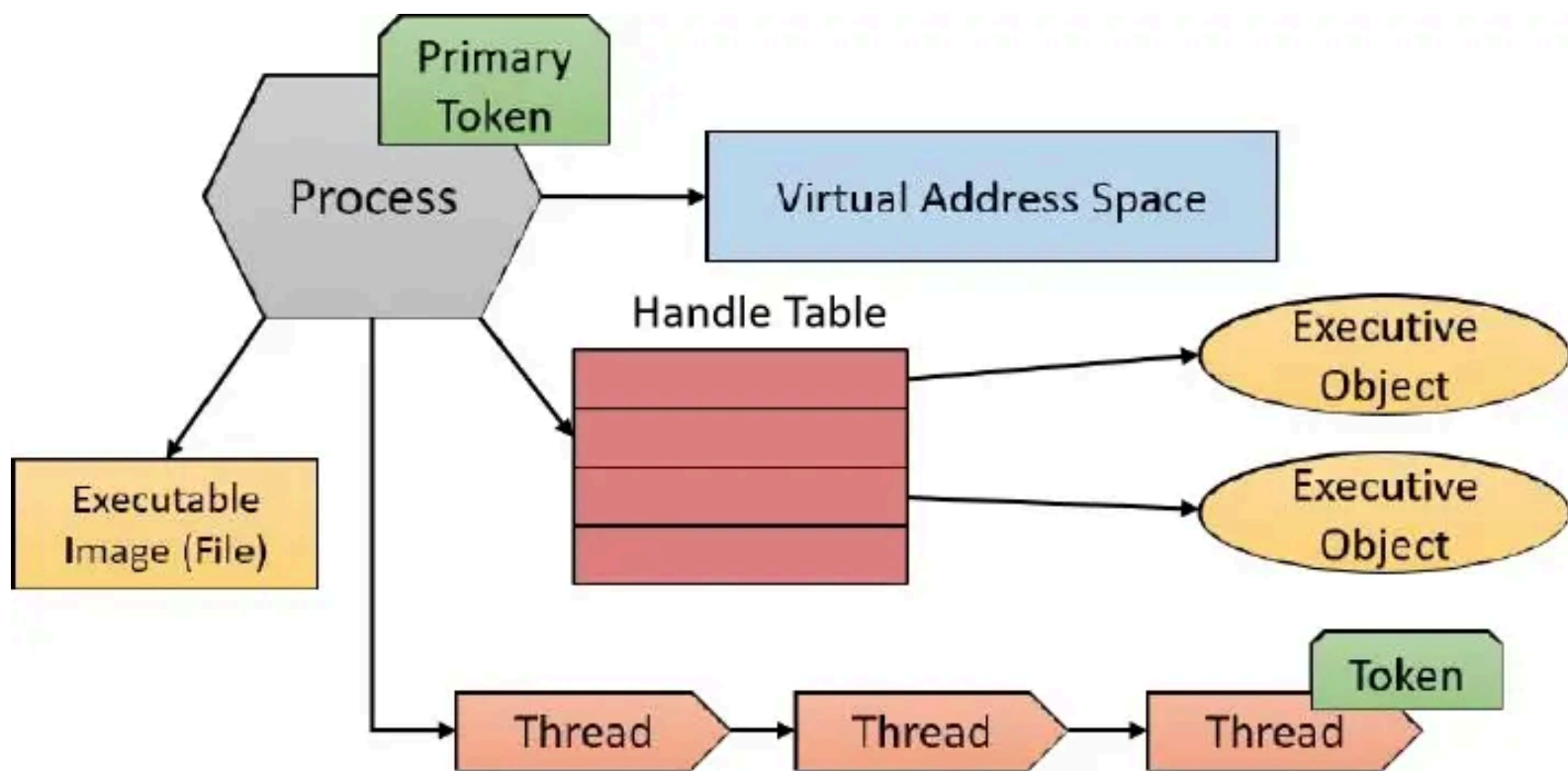
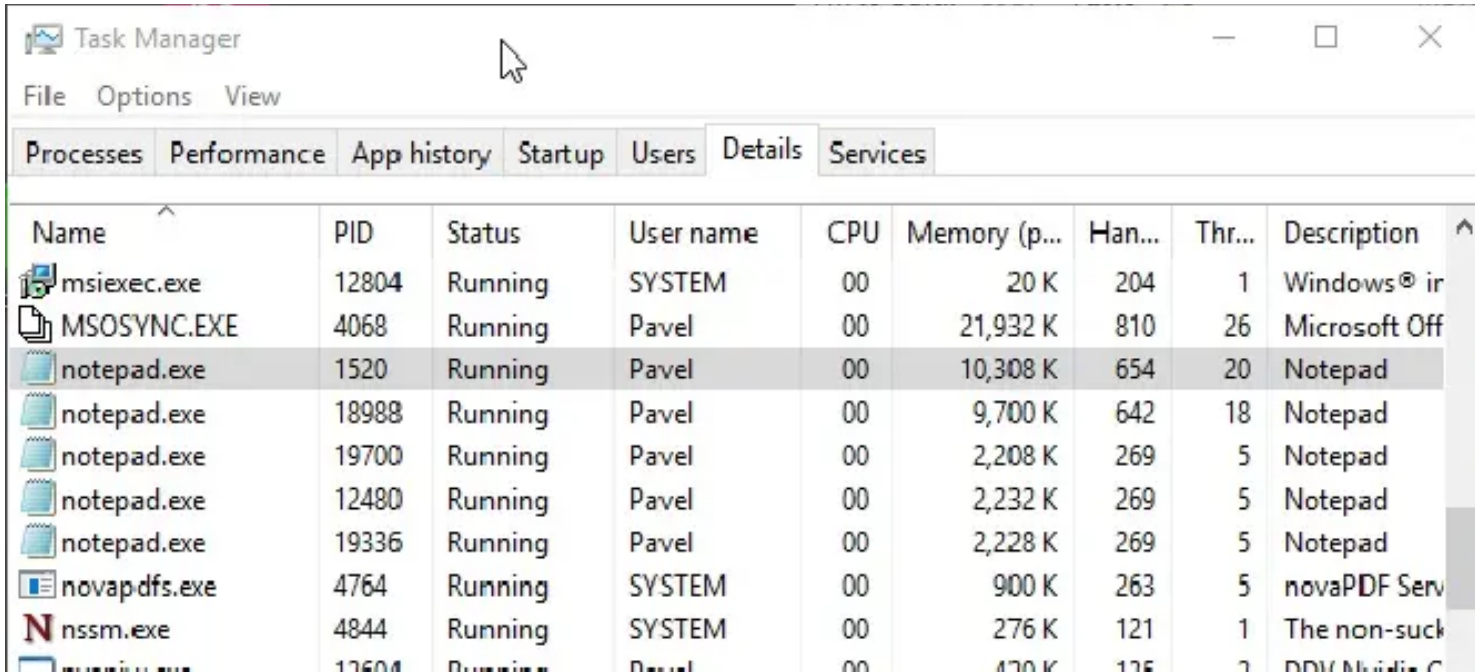


Figure 1-1: Important ingredients of a process

A process is uniquely identified by its Process ID, which remains unique as long as the kernel process object exists. Once it's destroyed, the same ID may be reused for new processes. It's important to realize that the executable file itself is not a unique identifier of a process. For example, there may be five instances of *notepad.exe* running at the same time. Each process has its own address space, its own threads, its own handle table, its own unique process ID, etc. All those five processes are using the same image file (*notepad.exe*) as their initial code and data. Figure 1-2 shows a screen shot of Task Manager's Details tab showing five instances of *Notepad.exe*, each with its own attributes.



The screenshot shows the Windows Task Manager window with the 'Processes' tab selected. A list of running processes is displayed with columns for Name, PID, Status, User name, CPU, Memory (p...), Han..., Thr..., and Description. Five instances of 'notepad.exe' are highlighted, showing they are all running under the user 'Pavel'.

Name	PID	Status	User name	CPU	Memory (p...	Han...	Thr...	Description
msiexec.exe	12804	Running	SYSTEM	00	20 K	204	1	Windows® ir
MSOSYNC.EXE	4068	Running	Pavel	00	21,932 K	810	26	Microsoft Off
notepad.exe	1520	Running	Pavel	00	10,308 K	654	20	Notepad
notepad.exe	18988	Running	Pavel	00	9,700 K	642	18	Notepad
notepad.exe	19700	Running	Pavel	00	2,208 K	269	5	Notepad
notepad.exe	12480	Running	Pavel	00	2,232 K	269	5	Notepad
notepad.exe	19336	Running	Pavel	00	2,228 K	269	5	Notepad
novapdfs.exe	4764	Running	SYSTEM	00	900 K	263	5	novaPDF Serv
nssm.exe	4844	Running	SYSTEM	00	276 K	121	1	The non-suck

Figure 1-2: Five instances of notepad

Dynamic Link Libraries

Dynamic Link Libraries (DLLs) are executable files that can contain code, data and resources (at least one of these). DLLs are loaded dynamically into a process either at process initialization time (called *static linking*) or later when explicitly requested (*dynamic linking*). We'll look at DLLs in more detail in chapter 15. DLLs don't contain a standard *main* functions like executables, and so cannot be run directly. DLLs allow sharing their code in physical memory between multiple processes that use the same DLL, which is the case for all standard Windows DLLs stored in the *System32* directory. Some of these DLLs, known as *subsystem DLLs* implement the documented Windows API, which is the focus of this book.

Figure 1-3 shows two processes using shared DLLs mapped to the same physical (and virtual) addresses.

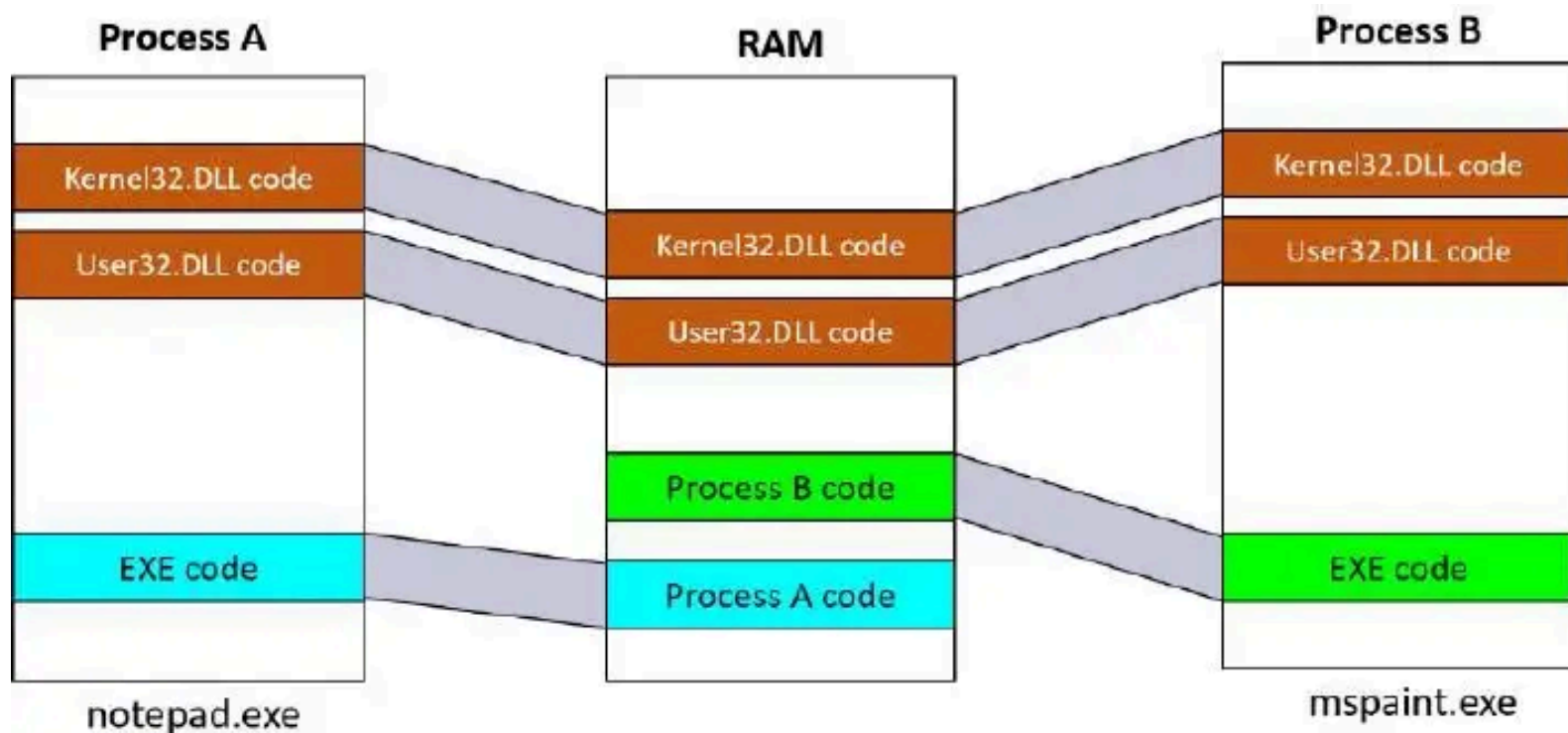


Figure 1-3: Sharing DLLs code

Virtual Memory

Every process has its own virtual, private, linear address space. This address space starts out empty (or close to empty, since the executable image and *NtDll.Dll* are normally the first to be mapped). Once execution of the main (first) thread begins, memory is likely to be allocated, more DLLs loaded, etc. This address space is private, which means other processes cannot access it directly. The address space range starts at zero (although technically the first 64KB of address cannot be allocated), and goes all the way to a maximum which depends on the process “bitness” (32 or 64 bit), the operating system “bitness” and a linker flag, as follows:

- For 32-bit processes on 32-bit Windows systems, the process address space size is 2 GB by default.
- For 32-bit processes on 32-bit Windows systems that use the increase user address space setting, that process address space size can be as large as 3 GB (depending on the exact setting). To get the extended address space range, the executable from which the process was created must have been marked with the `LARGEADDRESSAWARE` linker flag in its header. If it was not, it would still be limited to 2 GB.
- For 64-bit processes (on a 64-bit Windows system, naturally), the address space size is 8 TB (Windows 8 and earlier) or 128 TB (Windows 8.1 and later).

- For 32-bit processes on a 64-bit Windows system, the address space size is 4 GB if the executable image is linked with the `LARGEADDRESSAWARE` flag. Otherwise, the size remains at 2 GB.



The requirement of the `LARGEADDRESSAWARE` flag stems from the fact that a 2 GB address range requires 31 bits only, leaving the most significant bit (MSB) free for application use. Specifying this flag indicates that the program is not using bit 31 for anything and so setting that bit to 1 (which would happen for addresses larger than 2 GB) is not an issue.

The memory itself is called *virtual*, which means there is an indirect relationship between an address range and the exact location where it's found in physical memory (RAM). A buffer within a process may be mapped to physical memory, or it may temporarily reside in a file (such as a page file). The term virtual refers to the fact that from an execution perspective, there is no need to know if the memory about to be accessed is in RAM or not; if the memory is indeed mapped to RAM, the CPU will access the data directly. If not, the CPU will raise a page fault exception that will cause the memory manager's page fault handler to fetch the data from the appropriate file, copy it to RAM, make the required changes in the page table entries that map the buffer, and instruct the CPU to try again.

Threads

The actual entities that execute code are threads. A Thread is contained within a process, using the resources exposed by the process to do work (such as virtual memory and handles to kernel objects). The most important attributes a thread owns are the following:

- Current access mode, either user or kernel.
- Execution context, including processor registers.
- A stack, used for local variable allocations and call management.
- Thread Local Storage (TLS) array, which provides a way to store thread-private data with uniform access semantics.
- Base priority and a current (dynamic) priority.
- Processor affinity, indicating on which processors the thread is allowed to run on.

The most common states a thread can be in are:

- **Running** - currently executing code on a (logical) processor.

- **Ready** - waiting to be scheduled for execution because all relevant processors are busy or unavailable.
- **Waiting** - waiting for some event to occur before proceeding. Once the event occurs, the thread moves to the *Ready* state.

General System Architecture

Figure 1-4 shows the general architecture of Windows, comprising of user-mode and kernel-mode components.

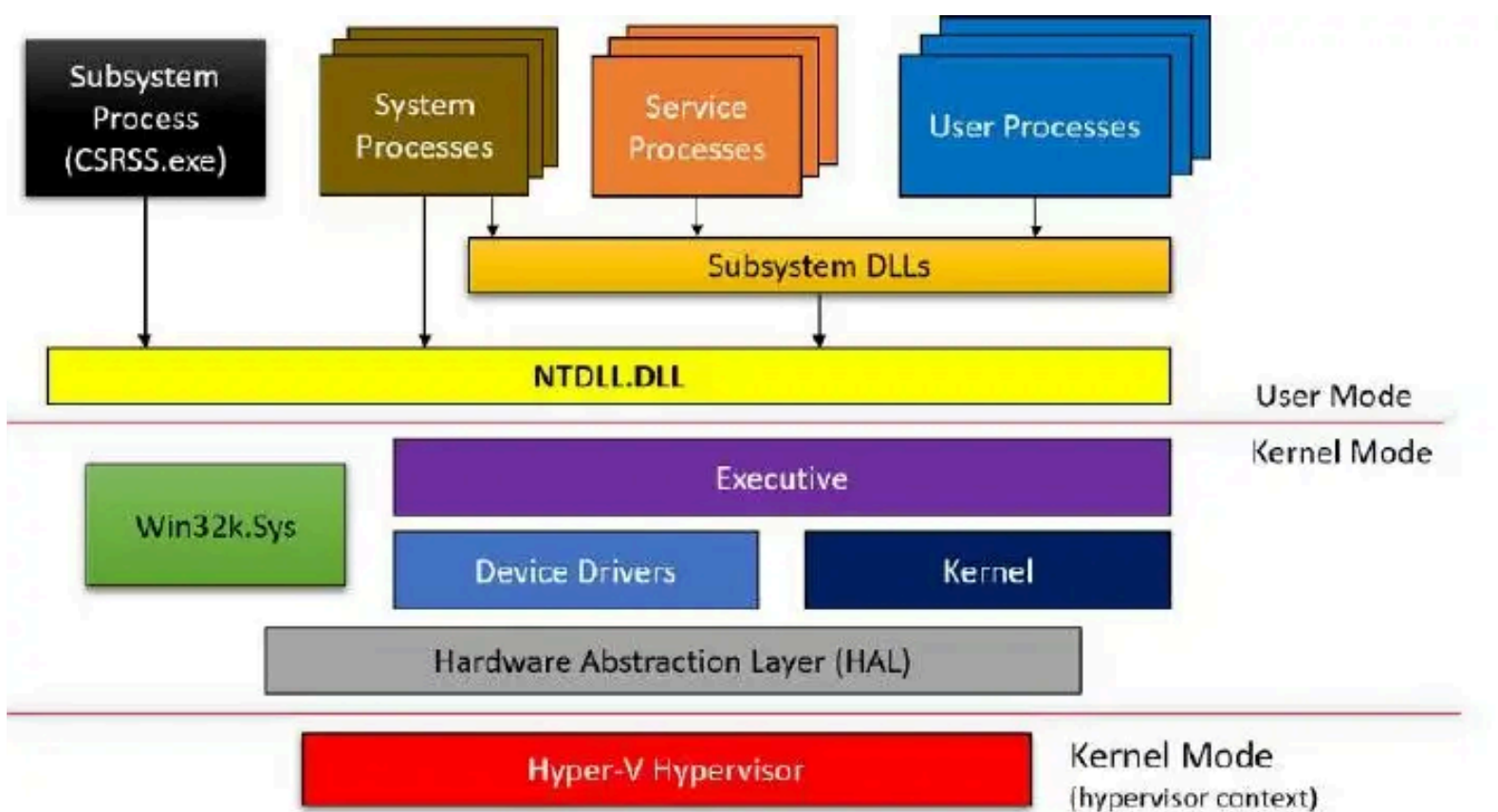


Figure 1-4: Windows system architecture

Here's a quick rundown of the named boxes appearing in figure 1-3:

- **User processes**

These are normal processes based on image files, executing on the system, such as instances of Notepad.exe, cmd.exe, explorer.exe and so on.

- **Subsystem DLLs**

Subsystem DLLs are dynamic link libraries (DLLs) that implement the API of a subsystem. A subsystem is a certain view of the capabilities exposed by the kernel. Technically, starting from Windows 8.1, there is only a single subsystem - the Windows Subsystem. The subsystem DLLs include well-known files, such as kernel32.dll,

user32.dll, gdi32.dll, advapi32.dll, combase.dll and many others. These include mostly the officially documented API of Windows. This book focuses on using the APIs exposed by these DLLs.

- **NTDLL.DLL**

A system-wide DLL, implementing the Windows native API. This is the lowest layer of code which is still in user mode. Its most important role is to make the transition to kernel mode for system call invocation. NTDLL also implements the Heap Manager, the Image Loader and some part of the user-mode thread pool. Although the native API is mostly undocumented, we will use some of it in this book where the standard documented Windows API is not available to achieve some goals.

- **Service Processes**

Service processes are normal Windows processes that communicate with the Service Control Manager (SCM, implemented in *services.exe*) and allow some control over their lifetime. The SCM can start, stop, pause, resume and send other messages to services. Chapter 19 deals with services in more detail.

- **Executive**

The Executive is the upper layer of *NtOskrnl.exe* (the “kernel”). It hosts most of the code that is in kernel mode. It includes mostly the various “managers”: Object Manager, Memory Manager, I/O Manager, Plug & Play Manager, Power Manager, Configuration Manager, etc. It’s by far larger than the lower Kernel layer.

- **Kernel**

The Kernel layer implements the most fundamental and time-sensitive parts of kernel mode OS code. This includes thread scheduling, interrupt and exception dispatching and implementation of various kernel primitives such as mutex and semaphore. Some of the kernel code is written in CPU-specific machine language for efficiency and for getting direct access to CPU-specific details.

- **Device Drivers**

Device drivers are loadable kernel modules. Their code executes in kernel mode and so has the full power of the kernel. Classic device drivers provide the glue between hardware devices and the rest of the OS. Other types of drivers provide filtering capabilities. For more information on device drivers, see my book “Windows Kernel Programming”.

- **Win32k.sys**

The *kernel-mode component of the Windows subsystem*. Essentially this is a kernel module (driver) that handles the user interface part of Windows and the classic Graphics Device Interface (GDI) APIs. This means that all windowing operations are handled by this component. The rest of the system has little-to-none knowledge of UI.

- **Hardware Abstraction Layer (HAL)**

The HAL is an abstraction layer over the hardware closest to the CPU. It allows device drivers to use APIs that do not require detailed and specific knowledge of things like an Interrupt Controller or a DMA controller. Naturally, this layer is mostly useful for device drivers written to handle hardware devices.

- **System Processes**

System processes is an umbrella term used to describe processes that are typically “just there”, doing their thing where normally these processes are not communicated with directly. They are important nonetheless, and some in fact, critical to the system’s well-being. Terminating some of them is fatal and causes a system crash. Some of the system processes are native processes, meaning they use the native API only (the API implemented by NTDLL). Example system processes include `Smss.exe`, `Lsass.exe`, `Winlogon.exe`, `Services.exe` and others.

- **Subsystem Process**

The Windows subsystem process, running the image `Csrss.exe`, can be viewed as a helper to the kernel for managing processes running under the Windows system. It is a critical process, meaning if killed, the system would crash. There is normally one `Csrss.exe` instance per session, so on a standard system two instances would exist - one for session 0 and one for the logged-on user session (typically 1). Although `Csrss.exe` is the “manager” of the Windows subsystem (the only one left these days), its importance goes beyond just this role.

- **Hyper-V Hypervisor**

The Hyper-V hypervisor exists on Windows 10 and server 2016 (and later) systems if they support *Virtualization Based Security* (VBS). VBS provides an extra layer of security, where the actual machine is in fact a virtual machine controlled by Hyper-V. VBS is beyond the scope of this book. For more information, check out the *Windows Internals* book.

Windows Application Development

Windows offers an *Application Programming Interface* (API) for use by developers to access a Windows’ system functionality. The classic API is known as the *Windows API*, and consists mostly of a long list of C functions, providing functionality from base services dealing with processes, threads and other low-level objects, to user interface, graphics, networking and everything in between. This book focuses mostly on using this API to program Windows.

Starting with Windows 8, Windows supports two somewhat distinct application types: the classic desktop applications that were the only application type prior to Windows 8 and Universal Windows Applications that can be uploaded to the Windows Store. From an internals perspective, these two types of applications are the same. Both types use threads, virtual memory, DLLs, handles, etc. Store applications primarily use the Windows Runtime APIs (described later in this section) but can use a subset of the classic Windows API. Conversely, desktop applications use the classic Windows API, but can also leverage a subset of the Windows Runtime APIs. This book focuses on desktop applications because the entire Windows API is available for them to use since this API contains the majority of functionality useful for system programming.

Other API styles offered by Windows, especially starting from Windows Vista are based on the *Component Object Model* (COM) technology - a component-oriented programming paradigm released in 1993 and used today by many components and services in Windows. Examples include DirectX, Windows Imaging Component (WIC), DirectShow, Media Foundation, Background Intelligent Transfer Service (BITS), Windows Management Instrumentation (WMI) and more. The most fundamentals concept in COM is the *interface* - a contract consisting of a collection of functions under a single container. We'll look at the basics of COM in chapter 18.

Naturally, over the years, various wrappers for these two fundamental API styles have been developed, some by Microsoft, some by others. Here are some of the common ones developed by Microsoft:

- *Microsoft Foundation Classes* (MFC) - C++ wrappers for (mostly) the user interface (UI) functionality exposed by Windows - working with windows, controls, menus, GDI, dialogs, etc.
- *Active Template Library* (ATL) - a C++ template-based library geared towards building COM servers and clients. We will use ATL in chapter 18 to simplify writing COM-related code.
- *Windows Template Library** (WTL) - an extension to ATL, providing template-based wrappers for Windows user interface functionality. It's comparable to MFC in terms of features, but is more lightweight and does not carry a (large) DLL with it (as MFC does). We'll use WTL in this book to simplify UI-related code, as UI is not the focus of this book.
- .NET - a framework and a runtime (*Common Language Runtime* - CLR) that provide a host of services, such as Just in Time (JIT) compilation of *Intermediate Language* (IL)

to native code and garbage collection. .NET can be used by leveraging new languages (C# being the most well-known) that provide lots of features, many of them abstracting Windows functionality and increasing productivity. The .NET framework uses the standard Windows APIs to accomplish its higher-level functionality. .NET is beyond the scope of this book - see the book “CLR Via C#” by Jeffrey Richter for an excellent coverage of .NET’s inner workings and functionality.

- *Windows Runtime* (WinRT) - this is the newest API layer added in Windows 8 and later versions. Its primary goal is developing applications based on the *Universal Windows Platform* (UWP). These applications can be packaged and uploaded to the Windows Store and downloaded by anyone. The Windows Runtime is built around an enhanced version of COM, so it too consists of interfaces as its primary (but not only) building block. Although this platform is native (and not based on .NET), it can be used by C++, C# (and other .NET languages) and even JavaScript - Microsoft provides *language projections* to simplify accessing the Windows Runtime APIs. A subset of the Windows Runtime APIs is available for (classic) desktop applications. We’ll take a look at the basics of the Windows Runtime in chapter 19.

Most of the standard Windows API function definitions are available in the `windows.h` header file. In some cases, additional headers will be needed, as well as additional import libraries. The text will point out any such headers and/or libraries.

Your First Application

This section describes the basics of using Visual Studio to write a simple application, compile and run it successfully. If you already know this, you can just skip this section.

First, you’ll need to install the proper tools to develop for Windows. Here is the short list of software in order:

1. Visual Studio 2017 or 2019, any edition, including the free community edition (available at <https://visualstudio.microsoft.com/downloads/>). Earlier versions of Visual Studio work just fine, but it’s usually best to stick with the latest versions, as these include compiler improvements as well as usability enhancements. In the installer’s main window, make sure at least the *Desktop Development with C++* workload is selected, as shown in figure 1-5.

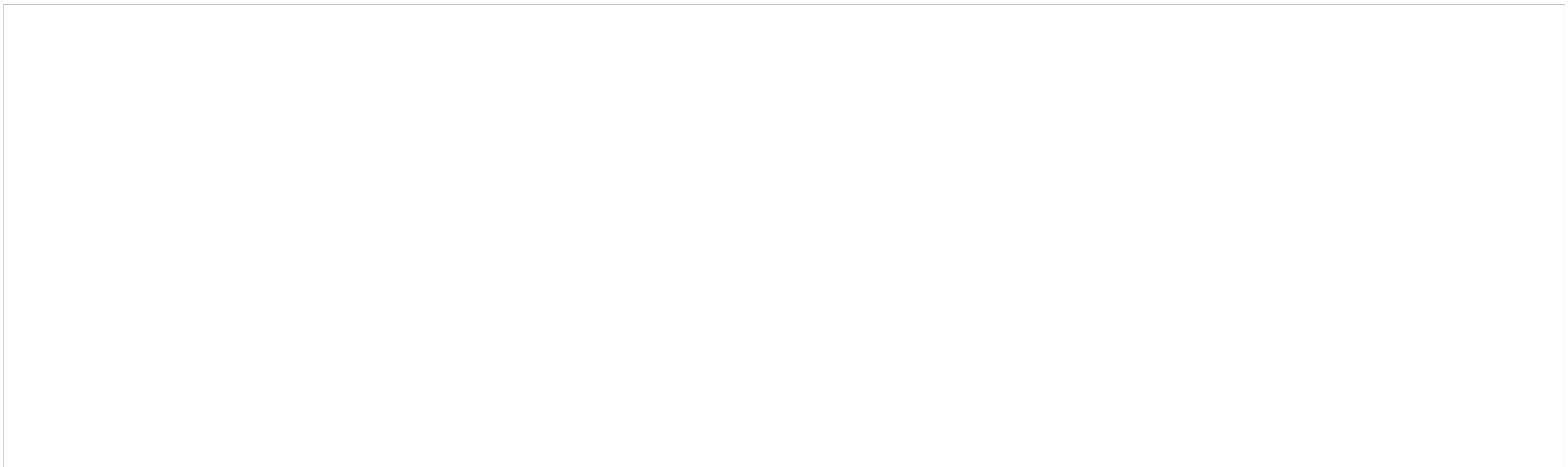


Figure 1-5: Visual Studio installer main window

2. The Windows *Software Development Kit* (SDK) is an optional installation that provides (possibly) updated headers and libraries, as well as various tools.

Once Visual Studio 2017/2019 is installed, run it and select to create a new project.

- In Visual Studio 2017, select *File / New Project...* from the menu and locate The C++ / **Desktop** node, and select the **Windows Console Application** project template, as shown in figure 1-6.
- In Visual Studio 2019, select *CreateNewProject* from the startup window and filter with **console** and C++ in the project type and language, respectively, and select **Console App** (make sure the language listed is C++). This is shown in figure 1-7.

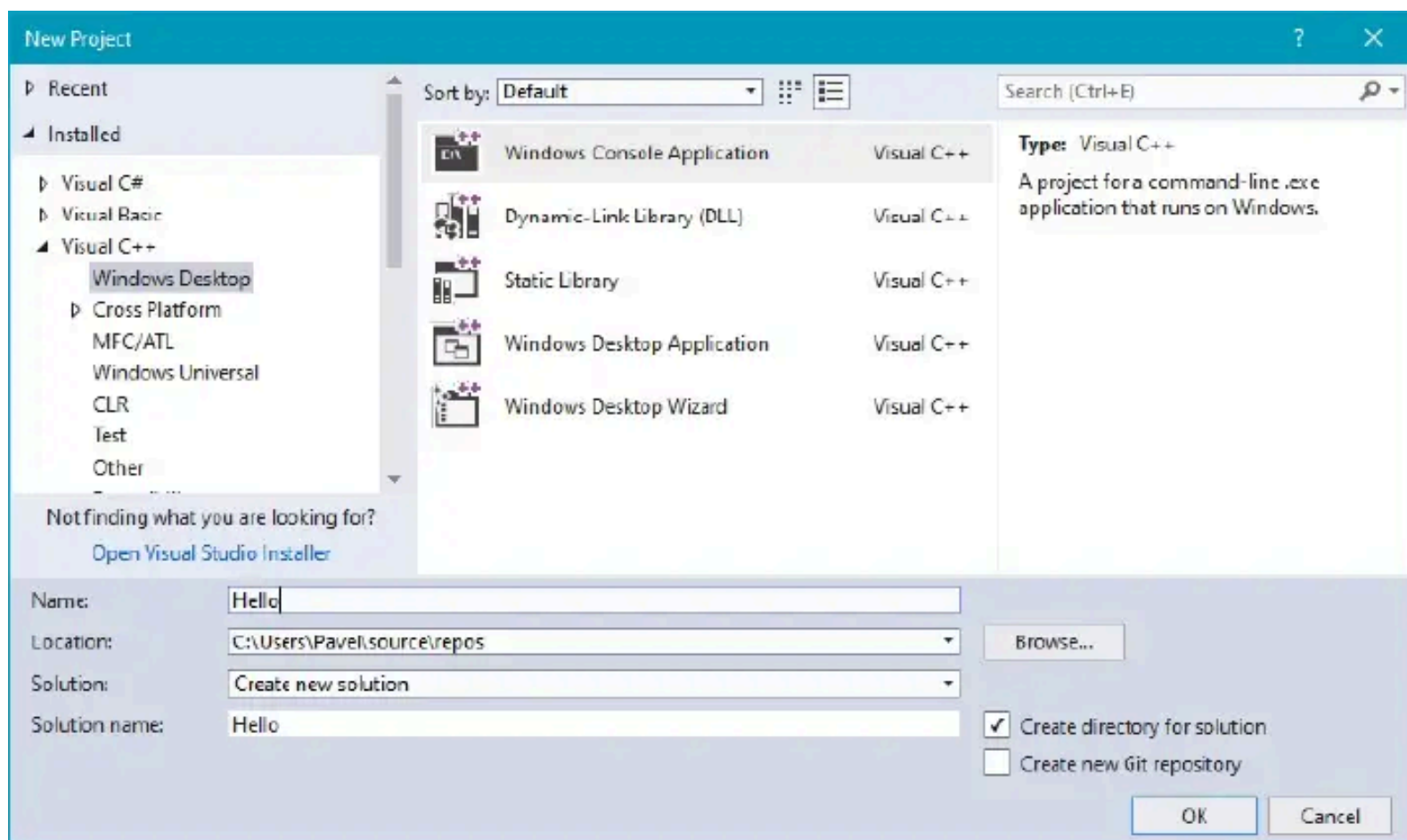


Figure 1-6: New project dialog in Visual Studio 2017

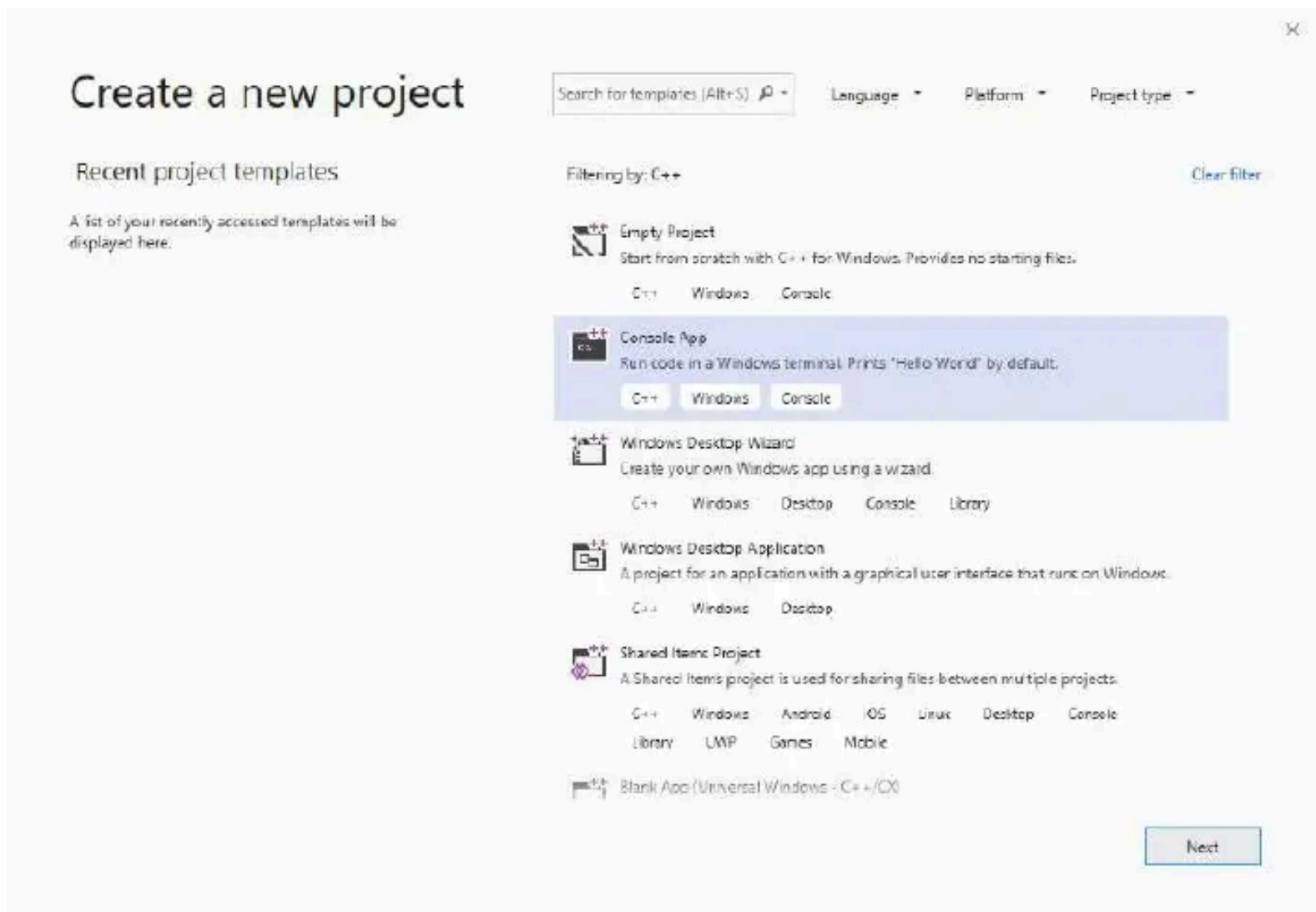


Figure 1-7: New project dialog in Visual Studio 2019

Name the project *HelloWin*, and change the destination folder if you wish, and click *OK*. The project should be created with a `HelloWin.cpp` file open in the editor with a minimal `main` function.

Add an `#include` at the top of the file for `windows.h`:

```
#include <windows.h>
```

If your project has a precompiled header (an `#include "pch.h"` is at the top of every C/C++ source file,

add the `windows.h` `#include` to this file, so that after the first compilation, subsequent compilations will be faster.

You can also include it in a C/C++ file if you prefer, but the include must follow the `pch.h` include.

Add another `#include` for `stdio.h` to gain access to the `printf` function:

```
#include <stdio.h>
```

In this first application, we'll get some system information by calling the `GetNativeSystemInfo` function.

Here is the main function code:

```
int main() {
    SYSTEM_INFO si;
    ::GetNativeSystemInfo(&si);

    printf("Number of Logical Processors: %d\n", si.dwNumberOfProcessors);
    printf("Page size: %d Bytes\n", si.dwPageSize);
    printf("Processor Mask: 0x%p\n", (PVOID)si.dwActiveProcessorMask);
    printf("Minimum process address: 0x%p\n", si.lpMinimumApplicationAddress);
    printf("Maximum process address: 0x%p\n", si.lpMaximumApplicationAddress);

    return 0;
}
```

From the *Build* menu, select *Build Solution* to compile and link the project (technically all projects in the solution). Everything should compile and link without errors. Hit *Ctrl+F5* to launch the executable without attaching to the debugger. (Or use the *Debug* menu and select *Run Without Debugging*. A console window should open showing output similar to the following:

```
Number of Logical Processors: 12
Page size: 4096 Bytes
Processor Mask: 0x00000FFF
Minimum process address: 0x00010000
Maximum process address: 0x7FFEFFFF
```



If you run the application by pressing *F5* (*Debug* menu, *Start Debugging*), the console window will appear and very quickly disappear when the application exits. Using *Ctrl+F5* adds a convenient “Press any key to continue” prompt which lets you view the console output before dismissing the window.

Visual Studio typically creates two solution platforms (*x86* and *x64*), which can be switched easily with the Solution Platforms combobox located in the main toolbar. By default, **x86** is selected which would produce the output above. If you switch the platform to **x64** and rebuild (assuming you're running an Intel/AMD 64 bit version of Windows of course), you'll get a slightly different output:

```
Number of Logical Processors: 12
Page size: 4096 Bytes
Processor Mask: 0x00000000000000FF
Minimum process address: 0x0000000000001000
Maximum process address: 0x00007FFFFFFF
```

The differences stem from the fact that 64-bit processes use pointers which are 8 bytes in size, while 32-bit processes use 4 bytes pointers. The address space address information from the `SYSTEM_INFO` structure is typed as pointers, so their sizes vary by process "bitness". We'll discuss 32-bit and 64-bit development in more detail in the section "32-bit vs. 64-bit Development" later in this chapter.

Don't worry about the meaning of the information presented by this small application (although some of it is self-explanatory). We'll look at these terms in later chapters.



The use of the double colon before the function name in the above code `::GetNativeSystemInfo` is to emphasize the fact the function is part of the Windows API and not some member function of the current C++ class. In this example it's obvious as there is no C++ class around, but this convention will be used throughout the book regardless (it also slightly speeds up the compiler function lookup). More coding conventions are described later in this chapter in the section, "Coding Conventions".

Working with Strings

In classic C, strings are not real types, but are just pointers to characters that end with a zero. The Windows API uses strings this way in many cases, but not all cases. The question

of encoding comes up when dealing with strings. In this section, we'll take a look at strings in general and how they are used in the Windows APIs.

In classic C, there was only a single type representing a character - `char`. The characters represented by `char` are 8 bit in size at most, where the first 7-bit values utilize the ASCII encoding. Today's systems must support multiple character sets from many languages, and these cannot fit into 8 bits. Thus, new encodings were created under the umbrella term **Unicode**, officially available online at <http://www.unicode.org>.

The Unicode Consortium defines several other character encodings. Here are the common ones:

- UTF-8 - the prevalent encoding used for web pages. This encoding uses one byte for Latin characters that are part of the ASCII set, and more bytes per character for other languages, such as Chinese, Hebrew, Arabic, and many others. This encoding is popular because if the text is mostly English, it's compact in size. In general, UTF-8 uses from one to four bytes for each character.
- UTF-16 - uses two bytes per character in most cases and encompasses all languages in just two bytes. Some more esoteric characters from Chinese and Japanese may require four bytes, but these are rare.
- UTF-32 - uses four bytes per character. The easiest to work with, but potentially the most wasteful.



UTF stands for *Unicode Transformation Format*.

UTF-8 may be the best when size matters, but from a programming standpoint, it's problematic because random access cannot be used. For example, to get to the 100th character in a UTF-8 string, the code needs to scan from the start of the string and work its way sequentially because there is no way to know where the 100th character may be. On the other hand, UTF-16 is much more convenient to work with programmatically (if we disregard the esoteric cases) because accessing the 100th character means adding 200 bytes to the string's start address.

UTF-32 is too wasteful and is rarely used.

Fortunately, Windows uses UTF-16 within its kernel, where each character is exactly 2 bytes. The Windows API follows suit and uses UTF-16 encoding as well, which is great as strings don't need to be converted when API calls eventually land in the kernel. However, there is a slight complication with the Windows API.

Parts of the Windows API were migrated from 16-bit Windows and Consumer Windows (Windows 95/98). These systems used ASCII as their primary way of working, which means the Windows API used ASCII strings rather than UTF-16. A further complication arises when double-byte encoding was introduced, where each character was one or two bytes in size, losing the advantage of random access.

The net result of all this is that the Windows API contains UTF-16 and ASCII functions for compatibility reasons. Since today the aforementioned systems do not exist, it's best to leave the one byte per character strings alone and use the UTF-16 functions only. Using the ASCII functions will cause the string to be converted to UTF-16 and then used with the UTF-16 function.

UTF-16 is also beneficial when interoperating with the .NET Framework, because .NET's string type stored UTF-16 characters only. This means passing a UTF-16 string to .NET does not require any conversion or copying.

Here is an example for the function `CreateMutex`, which if searched on the web will lead to one of two functions: `CreateMutexA` and `CreateMutexW`. The offline documentation gives this prototype:

```
HANDLE CreateMutex(  
    _In_opt_ LPSECURITY_ATTRIBUTES lpMutexAttributes,  
    _In_     BOOL                  bInitialOwner,  
    _In_opt_ LPCTSTR              lpName);
```

The `_In_opt_` and other similar annotations are called *Syntax Annotation Language* (SAL) and are used to convey metadata information to function and structure definitions. This may be useful for humans as well as static analysis tools. The C++ compiler currently ignores these annotations, but the static analyzer available in Visual Studio Enterprise editions uses it to detect potential errors before actually running the program.

For now let's concentrate on the last parameter which is a string pointer typed as `LPCTSTR`. Let's break it down:

L=Long P=Pointer C=Constant STR=String. The only mystery is the T in between. LPCTSTR is in fact a typedef with one of the following definitions:

```
typedef LPCSTR  LPCTSTR;    // const char* (UNICODE not defined)
typedef LPCWSTR LPCTSTR;    // const wchar_t* (UNICODE defined)
```

The term “long pointer” means nothing today. All pointers are the same size in a particular process (4 bytes in 32-bit processes and 8 bytes in 64-bit processes). The terms “long” and “short” (or “near”) are remnants from 16-bit Windows where such terms actually had different meanings. Also, the types LPCTSTR and similar have another equivalent - without the L - PCTSTR, PCWSTR, etc. These are generally preferred in source code.

The definition of the UNICODE compilation constant makes LPCTSTR expand to a UTF-16 string, and its absence to an ASCII string. This also means CreateMutex cannot be a function, because the C language does not allow function overloading, where a single function name may have multiple prototypes. CreateMutex is a macro, expanding to CreateMutexW (UNICODE defined) or CreateMutexA (UNICODE not defined). Visual Studio defines the UNICODE constant by default in all new projects, which is a good thing. We always want to use the UTF-16 functions to prevent the conversion from ANSI to UTF-16 (and of course for strings that contain non-ASCII characters, such a conversion is bound to be lossy).



W in CreateMutexW stands for *Wide* and A in CreateMutexA stands for *ANSI* or *ASCII*.

If the code needs to use a constant UTF-16 string, prefix the string with L to instruct the compiler to convert the string to UTF-16. Here are two versions of a string, one ASCII and the other UTF-16:

```
const char    name1[] = "Hello";    // 6 bytes (including NULL terminator)
const wchar_t name2[] = L"Hello";    // 12 bytes (including UTF-16 NULL terminator)
```

From this point on, we'll use the term "Unicode" to refer to UTF-16, unless otherwise explicitly stated.

Using the macros begs the question of how can we compile code that uses constant strings without explicitly choosing ASCII vs. Unicode? The answer lies with another macro, `TEXT`. Here is an example for `CreateMutex`:

```
HANDLE hMutex = ::CreateMutex(nullptr, FALSE, TEXT("MyMutex"));
```

The `TEXT` macro expands to the constant string with or without the "L" prefix depending on whether the `UNICODE` macro is defined or not. Since the ASCII functions are more expensive since they convert their values to Unicode before calling the wide functions, we should never use the ASCII functions. This means we can simply use the "L" prefix without the `TEXT` macro. We'll adopt this convention throughout the book.

There is a shorter version of the `TEXT` macro called `_T` defined in `<tchar.h>`. They are equivalent. Using these macros is still a fairly common practice, which is not bad in itself. However, I tend not to use it.

Similar to `LPCTSTR` there are other typedefs to allow using ASCII or Unicode, based on the `UNICODE` compilation constant. Table 1-1 shows some of these typedefs.

Table 1-1: Types used with strings

Common type(s)	ASCII type(s)	Unicode type(s)
TCHAR	char, CHAR	wchar_t, WCHAR
LPTSTR, PTSTR	char*, CHAR*, PSTR	wchar_t*, WCHAR*, PWSTR
LPCTSTR, PCTSTR	const char*, PCSTR	const wchar_t*, PCWSTR

Strings in the C/C++ Runtime

The C/C++ runtime has two sets of functions for manipulating strings. The classic (ASCII) ones begin with "str" such as `strlen`, `strcpy`, `strcat`, etc., but also Unicode versions

starting with “wcs”, such as `wcslen`, `wscpy`, `wscat` etc.

In the same vein as the Windows API, there is a set of macros that expand to either the ASCII or the Unicode version depending on another compilation constant, `_UNICODE` (notice the underscore). The prefix for these functions is “_tcs”. So we have functions named `_tcslen`, `_tcscpy`, `_tcscat`, etc., all working with the `TCHAR` type.

Visual Studio defines the `_UNICODE` constant by default so we get the Unicode functions if using the “_tcs” functions. It would be very weird if only one of the “UNICODE” constants would be defined, so avoid that.

String Output Parameters

Passing strings to functions as input as was done in the `CreateMutex` case is very common. Another common need is receiving results in the form of strings. The Windows API uses a few ways to pass back strings results.

The first (and the more common) case is where the client code allocates a buffer to hold the result string and provides the API with the size of the buffer (the maximum size the string can hold), and the API writes the string to the buffer up to the size specified. Some APIs also return the actual number of characters written and/or the number of characters required if the buffer is too small.

Consider the `GetSystemDirectory` function defined like so:

```
UINT GetSystemDirectory(
    _Out_ LPTSTR lpBuffer,
    _In_  UINT    uSize);
```

The function accepts a string buffer and its size and returns the number of characters written back. Notice all sizes are in **characters**, rather than bytes, which is convenient. The function returns zero in case of failure. Here is an example usage (error handling omitted for now):

```
WCHAR path[MAX_PATH];
::GetSystemDirectory(path, MAX_PATH);
printf("System directory: %ws\n", path);
```



Don't let the pointer type confuse you - the declaration of `GetSystemDirectory` does **not** mean you provide a pointer only. Instead, you must allocate a buffer and pass a pointer to this buffer.

MAX_PATH is defined in the Windows headers as 260, which is the standard maximum path in Windows (this limit can be extended starting with Windows 10, as we'll see in chapter 11). Notice that `printf` uses `%ws` as the string format to indicate it's a Unicode string, since UNICODE is defined by default and so all strings are Unicode.

The second common case is where the client code provides a string pointer only (via its address) and the API itself allocates the memory and places the resulting pointer in the provided variable. This means the client code is tasked with freeing the memory once the resulting string is no longer needed. The trick is to use the correct function to free the memory. The API's documentation indicates which function to use. Here is an example for this usage with the `FormatMessage` function defined like so (its Unicode variant):

```
DWORD FormatMessageW(
    _In_      DWORD dwFlags,
    _In_opt_  LPCVOID lpSource,
    _In_      DWORD dwMessageId,
    _In_      DWORD dwLanguageId,
    _When_((dwFlags & FORMAT_MESSAGE_ALLOCATE_BUFFER) != 0, _At__((LPWSTR*)lpBuf\
fer, _Outptr_result_z_))
    _When_((dwFlags & FORMAT_MESSAGE_ALLOCATE_BUFFER) == 0, _Out_writes_z_(nSiz\
e))
    LPWSTR lpBuffer,
    _In_      DWORD nSize,
    _In_opt_  va_list *Arguments);
```

Looks scary, right? I purposefully included the full SAL annotations for this function as the `lpBuffer` parameter is tricky. `FormatMessage` returns a string representation of an error number (we'll discuss errors in more detail in the section "API Errors" later in this chapter). The function is flexible in the sense that it can allocate the string itself or have the client provide a buffer to hold the resulting string. The actual behavior depends on the first `dwFlags` parameter: if it includes the `FORMAT_MESSAGE_ALLOCATE_BUFFER` flag, the function will allocate the buffer of the correct size. If the flag is absent, it's up to the caller to provide storage for the returned string.

All this makes the function a bit tricky, at least because if the former option is selected, the pointer type should be `LPWSTR*` - that is, a pointer to a pointer to be filled in by the function. This requires a nasty cast to make the compiler happy.

Here is a simple `main` function that accepts an error number from the command line arguments and shows its string representation (if any). It uses the option of letting the

function make the allocation. The reason is that there is no way to know what length the string should be, so it's best to let the function allocate the correct size.

```
int main(int argc, const char* argv[]) {
    if (argc < 2) {
        printf("Usage: ShowError <number>\n");
        return 0;
    }

    int message = atoi(argv[1]);

    LPWSTR text;
    DWORD chars = ::FormatMessage(
        FORMAT_MESSAGE_ALLOCATE_BUFFER |    // function allocates
        FORMAT_MESSAGE_FROM_SYSTEM |
        FORMAT_MESSAGE_IGNORE_INSERTS,
        nullptr, message, 0,
        (LPWSTR)&text,    // ugly cast
        0, nullptr);

    if (chars > 0) {
        printf("Message %d: %ws\n", message, text);
        ::LocalFree(text);
    }
    else {
        printf("No such error exists\n");
    }
    return 0;
}
```

The complete project is called *ShowError* in the Github repository for the book

Notice the call to the `LocalFree` function to free the string if the call is successful. The documentation for `FormatMessage` states that this is the function to call to free the buffer.

Here is an example run:

```
C:\Dev\Win10SysProg\x64\Debug>ShowError.exe 2
```

```
Message 2: The system cannot find the file specified.
```

```
C:\Dev\Win10SysProg\x64\Debug>ShowError.exe 5
```

```
Message 5: Access is denied.
```

```
C:\Dev\Win10SysProg\x64\Debug>ShowError.exe 129
```

```
Message 129: The %1 application cannot be run in Win32 mode.
```

```
C:\Dev\Win10SysProg\x64\Debug>ShowError.exe 1999
```

```
No such error exists
```

Safe String Functions

Some of the classic C/C++ runtime string functions (and some similar functions in the Windows API) are not considered “safe” from a security and reliability standpoint. For example, the `strcpy` function is problematic because it copies the source string to the target pointer until a NULL terminator is reached. This could overflow the target buffer and cause a crash in the good case (the buffer could be on the stack for example, and corrupt the return address stored there), and be used as a buffer overflow attack where an alternate return address is stored on the stack, jumping to a prepared shellcode.

To mitigate these potential vulnerabilities, a set of “safe” string functions were added to the C/C++ runtime library by Microsoft, where an extra parameter is used to specify the maximum size of a target buffer, so it will never overflow. These functions have a “_s” suffix, such as `strcpy_s`, `wscat_s`, etc.

Here are some examples using these functions:

```
void wmain(int argc, const wchar_t* argv[]) {  
    // assume arc >= 2 for this demo  
  
    WCHAR buffer[32];  
    wcscpy_s(buffer, argv[1]);           // C++ version aware of static buffers  
  
    WCHAR* buffer2 = (WCHAR*)malloc(32 * sizeof(WCHAR));  
    //wcscpy_s(buffer2, argv[1]);        // does not compile  
    wcscpy_s(buffer2, 32, argv[1]);      // size in characters (not bytes)
```

```
    free(buffer2);
}
```

The maximum size is always specified in characters and not bytes. Also note that these functions are able to calculate the maximum size automatically if the target buffer is statically allocated, which is convenient.

Another set of safe string functions was also added to the Windows API, at least for the purpose of reducing dependency on the C/C++ runtime. These functions are declared (and implemented) in the header `<strsafe.h>`. They are built according to the Windows API conventions, where the functions are actually macros expanding to functions with “A” or “W” suffix. Here are some simple examples of usage (using the same declarations as above):

```
StringCchCopy(buffer, _countof(buffer), argv[1]);
StringCchCat(buffer, _countof(buffer), L"cat");
```

```
StringCchCopy(buffer2, 32, argv[1]);
StringCchCat(buffer2, 32, L"cat");
```



“Cch” stands for *Count of Characters*.

Notice these functions don’t have a C++ variant that knows how to handle statically allocated buffers. The solution is to use the `_countof` macro that returns the number of elements in an array. Its definition is something like `sizeof(a)/sizeof(a[0])` given an array `a`.

Which set of functions should you use? It’s mostly a matter of taste. The important point is to avoid the classic, non-safe functions. If you do try to use them, you’ll get a compiler error like this:

```
error C4996: 'wcscpy': This function or variable may be unsafe. Consider using \
wcscpy_s instead. To disable deprecation, use _CRT_SECURE_NO_WARNINGS. See onli\
ne help for details.
```

Clearly, this error can be disabled by defining `_CRT_SECURE_NO_WARNINGS` before including the C/C++ headers, but it would be a bad idea. This macro exists to help maintain compatibility with old source code that probably should not be touched when compiled with recent compilers.

32-bit vs. 64-bit Development

Starting with Windows Vista, Windows has official 32 bit and 64 bit versions (there was a non-commercial 64-bit version of Windows XP as well). Starting with Windows Server 2008 R2, all server versions are 64-bit only. Microsoft removed the 32-bit server versions since servers typically need lots of RAM and large process address space, making 32-bit systems too limited for server work.

The programming model of 32 bit and 64 bit is identical from an API perspective. You should be able to compile to 32 or 64 bit just by selecting the required configuration in Visual Studio and hit Build. However, if the code should build successfully for both 32-bit and 64-bit targets, coding must be done carefully to use types correctly. In 64-bit, pointers are 8 bytes in size, whereas in 32-bit they are just 4 bytes. This change can lead to errors if a pointer's size is assumed to be of a certain value. For example, consider this cast operation:

```
void* p = ...;
int value = (int)p;
// do something with value
```

This code is buggy, since in 64-bit the pointer value is truncated to 4 bytes to fit into `int` (`int` is still 4 bytes in a 64-bit compilation as well). If such a cast is truly needed, an alternate type should be used instead - `INT_PTR`:

```
void* p = ...;
INT_PTR value = (INT_PTR)p;
// do something with value
```

`INT_PTR` means: “int the size of a pointer”. The Windows headers define several types like this one for this exact reason. Other types maintain their size regardless of the compilation “bitness”. Table 1-2 shows some examples of common types and their sizes.

Table 1-2: Common Windows types

type name	size (32 bit)	size (64 bit)	Description
ULONG_PTR	4 bytes	8 bytes	unsigned integer the size of a pointer
PVOID, void*	4 bytes	8 bytes	void pointer
any pointer	4 bytes	8 bytes	
BYTE, uint8_t	1 bytes	1 bytes	unsigned 8 bit integer
WORD, uint16_t	2 bytes	2 bytes	unsigned 16 bit integer
DWORD, ULONG, uint32_t	4 bytes	4 bytes	unsigned 32 bit integer
LONGLONG, __int64, int64_t	8 bytes	8 bytes	signed 64 bit integer
SIZE_T, size_t	4 bytes	8 bytes	unsigned integer sized as native integer

The differences between 32 bit and 64 bit go beyond type sizes. The address space of a 64-bit process is 128 TB (Windows 8.1 and later) compared to a mere 2 GB for 32-bit processes. On x64 systems (Intel/AMD), 32-bit processes can execute just fine thanks to a translation layer called WOW64 (*Windows on Windows 64*). We'll take a deeper look at this layer in chapter 12. This has several implications which are discussed in that chapter as well.

All sample applications in the book should build and run successfully on x86 and x64 equally well, unless stated otherwise explicitly. It's always best to build for both x86 and x64 during development and fix any issues that may arise.

In this book we won't cover ARM and ARM64 explicitly. All programs should build and run just fine on such systems (32 bit on ARM, 64 bit on ARM64), but I didn't have access to such systems, and so could not verify this in person.

Lastly, if code should be compiled in 64 bit only (or 32 bit only), the macro `_WIN64` is defined for 64-bit compilations. For example, we could replace the following line from *HelloWin*:

```
printf("Processor Mask: 0x%p\n", (PVOID)si.dwActiveProcessorMask);
```

with

```

#ifdef _WIN64
    printf("Processor Mask: 0x%016lX\n", si.dwActiveProcessorMask);
#else
    printf("Processor Mask: 0x%08X\n", si.dwActiveProcessorMask);
#endif

```

This would be somewhat clearer rather than using the %p format string which automatically expects 4 bytes in 32-bit processes and 8 bytes in 64-bit processes. This forced a cast to PVOID because dwActiveProcessorMask is of type DWORD_PTR and would generate a warning when used with %p.



A better option here is to specify %zu or %zX, which is used to format size_t values, equivalent to DWORD_PTR.

Coding Conventions

Having any coding conventions is good for consistency and clarity, but the actual conventions vary, of course. The following coding conventions are used in this book.

- Windows API functions are used with a double colon prefix. Example: ::CreateFile.
- Type names use Pascal casing (first letter is capital, and every word starts with a capital letter as well. Examples: Book, SolidBrush. The exception is UI related classes that start with a capital 'C'; this is for consistency with WTL.
- Private member variables in C++ classes start with an underscore and use camel casing (first letter is small and subsequent words start with capital letter). Examples: _size, _isRunning. The exception is for WTL classes where private member variable names start with m_. This is for consistency with ATL/WTL style.
- Variable names do not use the old Hungarian notation. However, there may be some occasional exceptions, such as an h prefix for a handle and a p prefix for a pointer.
- Function names follow the Windows API convention to use Pascal casing.
- When common data types are needed such as vectors, the C++ standard library is used unless there is good reason to use something else.
- We'll be using the *Windows Implementation Library* (WIL) from Microsoft, released in a Nuget package. This library contains helpful types for easier working with the Windows API. A brief introduction to WIL is in the next chapter.

- Some samples have a user interface. The book uses the *Windows Template Library* (WTL) to simplify UI-related code. You can certainly use some other library for UI such as MFC, Qt, straight Windows API, or even .NET libraries such as WinForms or WPF (assuming you know how to call native functions from .NET). UI is not the focus of this book. If you need more details on native Windows UI development, consult the classic “Programming Windows”, 6th edition, by Charles Petzold.

Hungarian notation uses prefixes to make variable names hint at their type. Examples: `szName`, `dwValue`. This convention is now considered obsolete, although parameter names and structure members in the Windows API use it a lot.

There are some more coding conventions used later in this book, which will be described when these become relevant.

C++ Usage

The code samples in this book make some use of C++. We won't be using any “complex” C++ features, but mostly features that enhance productivity, help with error avoidance. Here are the main C++ features we'll use:

- The `nullptr` keyword, representing a true NULL pointer.
- The `auto` keyword that allows type inference when declaring and initializing variables. This is useful to reduce clutter, save some typing, and focus on the important parts of the code.
- The `new` and `delete` operators.
- Scoped enums (`enum class`).
- Classes with member variables and functions.
- Templates, where they make sense.
- Constructors and destructors, especially for building RAII (*Resource Acquisition Is Initialization*) types. RAII will be discussed in greater detail in the next chapter.

Handling API Errors

Windows API function may fail for a variety of reasons. Unfortunately, the way a function indicates success or failure is not consistent across all functions. That said, there are very few cases, briefly described in table 1-3.

function return type	Success is ...	Failure is ...	How to get the error number
BOOL	not FALSE (0)	FALSE (0)	call
HANDLE	not NULL (0) and not INVALID_HANDLE_VALUE (-1)	0 or -1	GetLastError call GetLastError
void	cannot fail (usually)	None	Not needed, but in rare cases throws an SEH exception
LSTATUS or LONG	ERROR_SUCCESS (0)	greater than zero	return value is the error itself
HRESULT	greater or equal to zero, usually S_OK (0)	negative number	return value is the error itself
other	depends	depends	look up function documentation

The most common case is returning a `BOOL` type. The `BOOL` type is not the same as the C++ `bool` type; `BOOL` is in fact a 32-bit signed integer. A non-zero return value indicates success, while a returned zero (`FALSE`) means the function has failed. It's important not to test against the `TRUE` (1) value explicitly, since a success can sometimes return a value different from one. If the function fails, the actual error code is available by calling `GetLastError`, responsible for storing the last error from an API function occurring on the current thread. Put another way, each thread has its own last error value, which makes sense in a multi-threaded environment like Windows - multiple threads may call API functions at the same time.

Here is an example of handling such an error:

```

BOOL success = ::CallSomeAPIThatReturnsBOOL();
if(!success) {
    // error - handle it (just print it in this example)
    printf("Error: %d\n", ::GetLastError());
}

```

The second item from table 1-3 is for functions returning `void`. There are actually very few such functions, and most cannot fail. Unfortunately, there are very few such functions that can actually fail in extreme circumstances (“extreme” usually means very low memory resources) with a *Structured Exception Handling* (SEH) exception. We’ll discuss SEH in

chapter 20. You probably don't need to worry too much about such functions, because if one of these does fail, it means the entire process and possibly the system is in big trouble anyway.

Next, there are functions returning `LSTATUS` or `LONG`, both of which are just signed 32-bit integers. The most common APIs using these scheme are the registry functions we'll meet in chapter 17. These functions return `ERROR_SUCCESS` (0) if successful. Otherwise, the returned value is the error itself (calling `GetLastError` is not needed).

Next on the list from table 1-3 is the `HRESULT` type, which is yet again a signed 32-bit integer. This return type is common for *Component Object Model* (COM) functions (COM is discussed in chapter 18). Zero or a positive value indicates success, while a negative value indicates an error, identified by the returned value. In most cases, checking for success or failure is done with the `SUCCEEDED` or `FAILED` macros, respectfully, returning just `true` or `false`. In rare cases the code would need to look at the actual value.

The Windows headers contain a macro to convert a Win32 error (`GetLastError`) to an appropriate `HRESULT`: `HRESULT_FROM_WIN32`, which is useful if a COM method needs to return an error based on a failed `BOOL`-returning API.

Here is an example for handling an `HRESULT` based error:

```
IGlobalInterfaceTable* pGit;
HRESULT hr = ::CoCreateInstance(CLSID_StdGlobalInterfaceTable, nullptr, CLSCTX_
ALL,
    IID_IGlobalInterfaceTable, (void**)&pGit);
if(FAILED(hr)) {
    printf("Error: %08X\n", hr);
}
else {
    // do work
    pGit->Release();    // release interface pointer
}
```

Don't worry about the details of the above code. Chapter 21 is dedicated to COM.

The last entry in table 1-3 is for "other" functions. For example, the `FormatMessage` function we met a few sections ago returns a `DWORD` indicating the number of characters copied to the

provided buffer, or zero if the function fails. There are no hard and fast rules for these kinds of functions - the documentation is the best guide. Luckily, there are not that many of them.

Defining Custom Error Codes

The error code mechanism exposed by `GetLastError` can be used by applications as well to set error codes in a similar vein. This is accomplished by calling `SetLastError` with the error to set on the current thread. A function can use one of the many predefined error codes, or it can define its own error codes. To prevent any collision with system-defined codes, the application should set bit 29 in the defined error code.

Here is an example of a function that uses this technique:

```
#define MY_ERROR_1 ((1 << 29) | 1)
#define MY_ERROR_2 ((1 << 29) | 2)

BOOL SomeApi1(int32_t, int32_t*);
BOOL SomeApi2(int32_t, int32_t*);

bool DoWork(int32_t value, int32_t* result) {
    int32_t result1;
    BOOL ok = ::SomeApi1(value, &result1);
    if (!ok) {
        ::SetLastError(MY_ERROR_1);
        return false;
    }

    int32_t result2;
    ok = ::SomeApi2(value, &result2);
    if (!ok) {
        ::SetLastError(MY_ERROR_2);
        return false;
    }

    *result = result1 + result2;
    return true;
}
```

Note that in my functions, I'm free to use the C++ `bool` type which can be true or false,

rather being a 32-bit integer (BOOL). The custom-defined error codes have bit 29 set, making sure they don't clash with a system defined error code.

The Windows Version

In some cases it's desirable to query the system for the Windows OS version on which the current application is executing. The official version numbers of Windows releases is shown in table 1-4.

Table 1-4: Windows version numbers

Windows release name	Official version number
Windows NT 3.1	3.1
Windows NT 3.5	3.5
Windows NT 4.0	4
Windows 2000	5.0
Windows XP	5.1
Windows Server 2003	5.2
Windows Vista / Server 2008	6.0
Windows 7 / Server 2008 R2	6.1
Windows 8 / Server 2012	6.2
Windows 8.1 / Server 2012 R2	6.3
Windows 10 / Server 2016	10.0

You may be wondering why the version numbers have these values - we'll get to that in a moment. The classic function to get this information is `GetVersionEx`, declared like so:

```
typedef struct _OSVERSIONINFO {
    DWORD dwOSVersionInfoSize;
    DWORD dwMajorVersion;
    DWORD dwMinorVersion;
    DWORD dwBuildNumber;
    DWORD dwPlatformId;
    TCHAR szCSDVersion[ 128 ];    // Maintenance string for PSS usage
} OSVERSIONINFO, *POSVERSIONINFO, *LPOSVERSIONINFO;

BOOL GetVersionEx(
    _Inout_ POSVERSIONINFO pVersionInformation);
```

Using it is fairly straightforward:

```
OSVERSIONINFO vi = { sizeof(vi) };
::GetVersionEx(&vi);

printf("Version: %d.%d.%d\n",
       vi.dwMajorVersion, vi.dwMinorVersion, vi.dwBuildNumber);
```

However, compiling it with recent SDKs causes a compilation error: “error C4996: ‘GetVersionExW’: was declared deprecated”.

The reason will become clear shortly. It’s possible to remove this deprecation by adding the following definition before including *<windows.h>*:

```
#define BUILD_WINDOWS
#include <Windows.h>
```

Running the above code snippet on Windows up to 8 (inclusive) gives back the correct Windows version. However, running it on Windows 8.1 or 10 (and their server equivalents), will always display the following output:

```
Version: 6.2.9200
```

This is the Windows version for Windows 8. Why? This was a defensive mechanism devised by Microsoft after some issues applications had on Windows Vista. Since Vista came out on January 2006, almost five years after Windows XP, many applications were built in the XP days, and some went to the trouble of checking the minimum Windows version to be XP using the following code:

```
OSVERSIONINFO vi = { sizeof(vi) };
::GetVersionEx(&vi);

if(vi.dwMajorVersion >= 5 && vi.dwMinorVersion >= 1) {
    // XP or later: good to go?
}
```

This code is buggy, because it doesn’t foresee the possibility of having a major number of 6 or higher with a minor version of zero. So, for Vista, the above condition failed and would notify the user “Please use XP or later”. The correct check would have been this:

```
if(vi.dwMajorVersion > 5 || (vi.dwMajorVersion == 5 && vi.dwMinorVersion >= 1) {  
    // XP or later: good to go!  
}
```

Unfortunately, too many applications had that bug, and so Microsoft decided for Windows 7 not to increase the major version number but only increment the minor number to 1; this solved the bug. What about Windows 8? Microsoft was still afraid of the above bug and so incremented the minor number only, giving 6.2. The reasoning for Windows 8.1 was similar (6.3). But what about Windows 10? Should the version be 6.4? This seems like a total defeat - how long can Microsoft continue leaving the major version as 6? Well, Windows 10 has version number 10.0. Does that mean all is well? Not really. As we saw, the call to `GetVersionEx` returns the Windows 8 numbers even on Windows 10. What gives?

A new feature has been introduced (called *Switchback*) that returned the Windows version as no higher than 8 (6.2) to prevent compatibility issues, *unless* the application in question has declared its knowledge of a higher-version Windows in existence. This is accomplished using a manifest file - an optional XML file with configuration information - that can be used to indicate a specific Windows version awareness, from Vista to 10.

This is not just for manipulating the returned version number, but also for some behavioral changes of some APIs for compatibility. This is accomplished using *Shims*, which changes API behavior depending on the selected OS version.

In Visual Studio, a manifest can be added by following these steps:

- Add an XML file to the project with a name like *manifest.xml*. This will hold the manifest file's contents.
- Fill in the manifest (shown after this list).
- Open *Project/Properties* and navigate to the *Manifest Tool* node, *Input and Output*. In *Additional Manifest Files*, type the name of the manifest file (figure 1-8).
- Build the project normally.

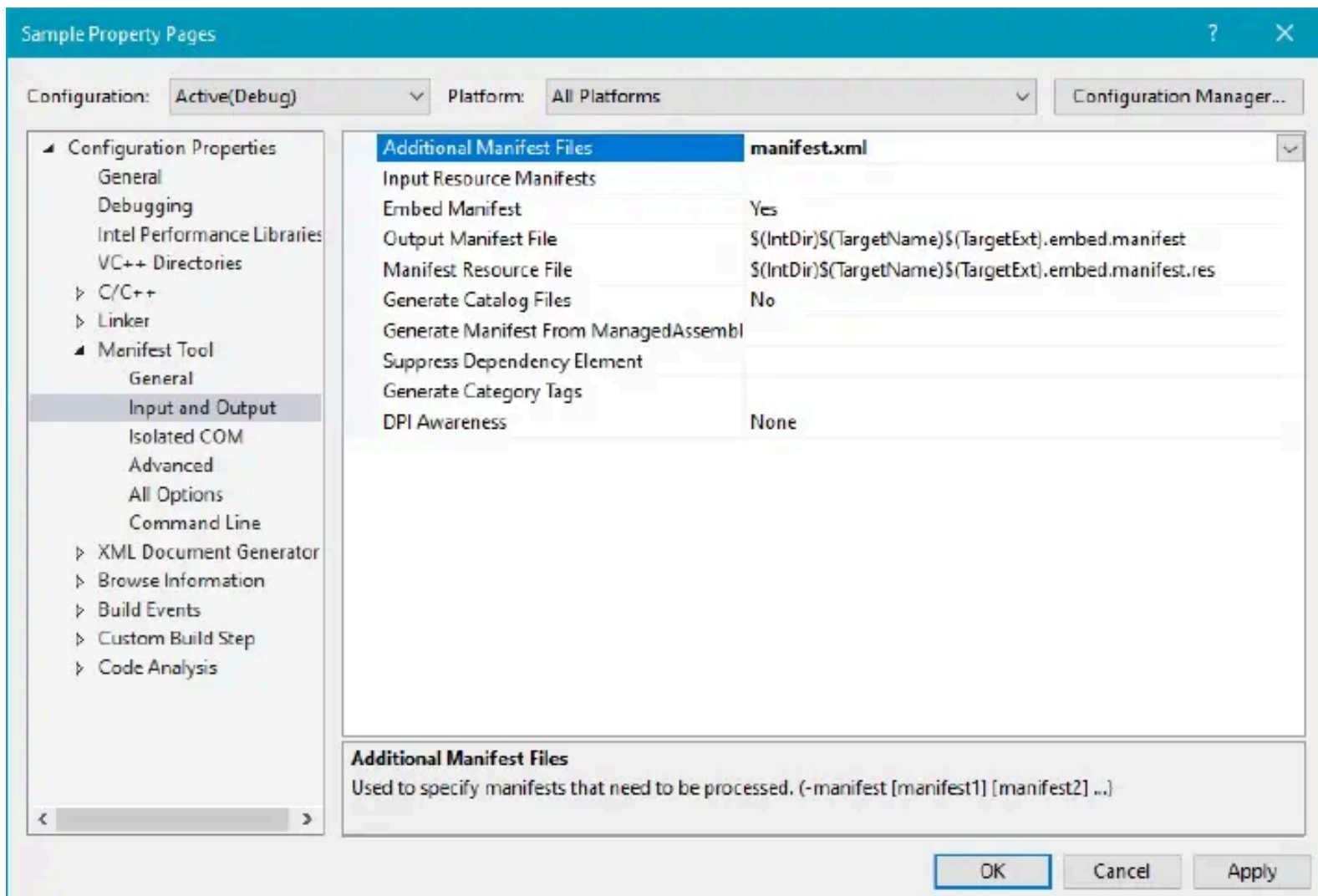


Figure 1-8: Setting manifest file

Notice the setting *Embed Manifest* = *Yes* in figure 1-8. This embeds the manifest as a resource in the executable rather than leaving it as a loose file in the same directory as the executable and always named {exename}.exe.manifest.

The manifest can have several elements, but we focus on just one in this chapter (we'll examine others in due course). Here it is:

```
<?xml version="1.0" encoding="utf-8"?>
<assembly manifestVersion="1.0" xmlns="urn:schemas-microsoft-com:asm.v1">
  <compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">
    <application>
      <!-- Windows Vista -->
      <!--<supportedOS Id="{e2011457-1546-43c5-a5fe-008deee3d3f0}" />-->

      <!-- Windows 7 -->
      <!--<supportedOS Id="{35138b9a-5d96-4fbd-8e2d-a2440225f93a}" />-->
    </application>
  </compatibility>
</assembly>
```

```

<!-- Windows 8 -->
<!--<supportedOS Id="{4a2f28e3-53b9-4441-ba9c-d69d4a4a6e38}" />-->

<!-- Windows 8.1 -->
<!--<supportedOS Id="{1f676c76-80e1-4239-95bb-83d0f6d0da78}" />-->

<!-- Windows 10 -->
<!--<supportedOS Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}" />-->
</application>
</compatibility>
</assembly>

```



The easiest way to get a nice manifest file to tweak is (ironically perhaps) create a simple console C# application, then add to the project an *Application Manifest File* item, which will generate the above XML, among other elements.

The GUIDs for the various OS versions have been created when those versions were released. This means there is no way an application developed in the Windows 7 days could get a version of Windows 10, for example.

If you uncomment the Windows 8.1 version for instance and re-run the application, the output would be:

```
Version: 6.3.9600
```

If you uncomment the Windows 10 GUID (whether the Windows 8.1 GUID is commented out or not is unimportant), you'll get the real Windows 10 version (if running on a Windows 10 machine, of course):

```
Version: 10.0.18362
```

Getting the Windows Version

Given that `GetVersionEx` is deprecated (at least for the reasons discussed in the previous section), what is the proper way to get the Windows version? A new set of APIs is available that can give back the result, but not in a simple numerical sense, but by returning true/false for Windows version questions. These are available in the `<versionhelpers.h>` header file.

Here are some of the functions included: `IsWindowsXPOrGreater`, `IsWindowsXPSP3OrGreater`, `IsWindows7OrGreater`, `IsWindows8Point1OrGreater`, `IsWindows10OrGreater`, `IsWindowsServer`. Their usage is straightforward - they accept nothing and return `TRUE` or `FALSE`. Their implementation uses another version-related function, `VerifyVersionInfo`:

```
BOOL VerifyVersionInfo(
    _Inout_ POSVERSIONINFOEX pVersionInformation,
    _In_     DWORD dwTypeMask,
    _In_     DWORDLONG dwlConditionMask);
```

This function knows how to compare version numbers based on the specified criteria (`dwlConditionMask`), such as the major or minor version numbers. You can find the implementation of all the Boolean functions inside `versionhelper.h`.

There is an undocumented (but reliable) way to get the version numbers regardless of the manifest file without calling `GetVersionEx`. It's based on a data structure called `KUSER_SHARED_DATA` that is mapped to every process to the same virtual address (`0x7FFE0000`). Its declaration is listed in this Microsoft link: https://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/content/ntddk/ns-ntddk-kuser_shared_data. The Windows version numbers are part of this shared structure in the same offsets. Here is an alternative to showing the Windows version numbers:

```
auto sharedUserData = (BYTE*)0x7FFE0000;
printf("Version: %d.%d.%d\n",
    *(ULONG*)(sharedUserData + 0x26c), // major version offset
    *(ULONG*)(sharedUserData + 0x270), // minor version offset
    *(ULONG*)(sharedUserData + 0x260)); // build number offset (Windows 10)
```

Of course, it's recommended to use the official APIs rather than `KUSER_SHARED_DATA`.

Exercises

1. Write a console application that prints more information about the system than the *HelloWin* application shown earlier, by calling the following APIs: `GetNativeSystem-`

Info, GetComputerName, GetWindowsDirectory, QueryPerformanceCounter, GetProductInfo, GetComputerObjectName. Handle errors if they occur.

Summary

In this chapter, we looked at the fundamentals of Windows, both in terms of architecture and in terms of programming. In the next chapter, we'll dig into kernel objects and handles, as these form the basis of many aspects of working with Windows.